Solana On/Off Ramp Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Farouk

JesJupyter

Ctrus

February 19, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **3**

6 Executive Summary **4**

7 Findings **7**

7.1 Critical Risk . . . 7

7.1.1 Investors can mint DsTokens out of thin air . . . 7

7.2 Medium Risk . . . 9

7.2.1 Unvalidated Payer in DS Token Issue CPI Allows Protocol or Registrar to Pay Creation Fees . 9

7.3 Low Risk . . . 11

7.3.1 DS Token Subscription Can Proceed Without Wallet Identity Verification When Registration Accounts Are Omitted . . . 11

7.3.2 Operator Swap Signatures Are Replayable and Not Bound to a Specific On-Ramp or Mints . 12

7.3.3 Initialization Allows Omitted `asset_vault` Although Swaps Require It . . . 12

7.3.4 Missing Executable Check for NAV Provider in `update_nav_provider` . . . 13

7.3.5 Inadequate validations on `collector-token-account`. . . . 14

7.3.6 Investors can grief Operators . . . 14

7.3.7 Countries restriction defaults to empty on initialize, allowing all jurisdictions to use the protocol 16

7.3.8 Allowance Liquidity Provider Uses `delegated_amount` Without Capping by Actual Balance . . 17

7.3.9 Incomplete Rate CPI Context and Understated Minimum NAV Provider Accounts . . . 17

7.4 Informational . . . 19

7.4.1 On-Ramp `initialize` Should Reject Zero Genesis Hash . . . 19

7.4.2 Redundant `#[instruction(asset_provider)]` Attribute in On-Ramp `UpdateAssetProvider` 19

7.4.3 Incompatible NavProviders can be set, which could fail swaps temporarily . . . 20

7.4.4 Off-Ramp `update_fee_manager` Does Not Validate `liquidity_mint` . . . 21

7.4.5 Insufficient checks on liquidity amount wastes compute unnecessary . . . 21

7.4.6 Investor's dsTokens May Become Locked After His Country Is Banned . . . 22

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Securitize Solana On-/Off-Ramp protocol is a regulated real-world asset (RWA) infrastructure system designed to provide compliant entry and exit paths between stablecoin liquidity (USDC) and tokenized assets on Solana. The protocol enables primary subscriptions (issuance), secondary-style swaps, and redemptions for both DS Protocol tokens (regulated RWAs) and frozen-by-default SPL tokens (public equities), serving as the on-chain bridge between fiat-denominated stablecoins and tokenized securities.

The system operates through two core Solana programs built on the Anchor framework. The on-ramp program converts stablecoin deposits into tokenized assets: an investor submits USDC, the program queries an external NAV provider for the current asset rate, calculates the corresponding asset amount after fees, splits the liquidity between a fee collector and a custodian wallet, and supplies asset tokens to the investor via one of three provider mechanisms (allowance-based transfer, direct minting, or SPL Access Control minting). The off-ramp program performs the inverse: a redeemer submits asset tokens, the program calculates the equivalent liquidity amount at the current NAV rate, handles the asset tokens according to policy (burn or transfer to a designated recipient), and supplies stablecoin liquidity from a delegated allowance provider. Fee calculation uses ceiling division with a configurable basis-points numerator capped at 50%. Both programs support a two-step transfer mode that routes tokens through an intermediate vault before final delivery.

The protocol enforces two distinct compliance models depending on the token standard. DS Token flows rely on the RWA Token suite (asset controller, identity registry, and policy engine) for transfer hook enforcement, investor identity verification, and country-based redemption restrictions. SPL Token flows use a different trust model: an authorized operator submits transactions on behalf of investors, including an Ed25519 signature from the investor over the swap or redemption parameters, with a short deadline (one to two minutes) for replay protection and a genesis hash for cross-cluster domain separation. DS Token swaps are self-service (investor signs directly), while SPL Token swaps require operator mediation.

Each program instance is governed by a single admin key that controls all configuration: pausing and unpausing operations, managing up to ten operators, updating fee parameters, swapping NAV and asset providers, toggling two-step transfers, and (on the off-ramp) managing country restriction bitmaps. There is no multi-signature requirement, timelock, or on-chain governance. The on-ramp restricts deployment to a hardcoded allowlist of initializer addresses, and both programs use PDA-derived authorities for token operations.

The security of the system depends on correct validation of all accounts passed through remaining accounts in CPI calls, faithful NAV rate reporting by external provider programs, the integrity of operator keys and the short-lived Ed25519 signature window for SPL token flows, and the assumption that the single admin key is not compromised.

5 Audit Scope

The Solana Off-Ramp component was previously audited by Cyfrin; the initial focus of this audit was the Solana On-Ramp.

During the audit the Off-Ramp component received some additional changes hence the audit was extended to also audit the new changes to the Off-Ramp component. The final scope limited to:

```
// new on-ramp component
programs/securitize-on-ramp/src/instructions/admin/add_operator.rs
programs/securitize-on-ramp/src/instructions/admin/change_admin.rs
programs/securitize-on-ramp/src/instructions/admin/initialize.rs
programs/securitize-on-ramp/src/instructions/admin/pause.rs
programs/securitize-on-ramp/src/instructions/admin/revoke_operator.rs
programs/securitize-on-ramp/src/instructions/admin/toggle_investor_subscription.rs
programs/securitize-on-ramp/src/instructions/admin/toggle_two_step_transfer.rs
programs/securitize-on-ramp/src/instructions/admin/unpause.rs
programs/securitize-on-ramp/src/instructions/admin/update_asset_provider.rs
programs/securitize-on-ramp/src/instructions/admin/update_fee_manager.rs
programs/securitize-on-ramp/src/instructions/admin/update_min_subscription_amount.rs
programs/securitize-on-ramp/src/instructions/admin/update_nav_provider.rs
programs/securitize-on-ramp/src/instructions/investor/subscribe_ds_token.rs
programs/securitize-on-ramp/src/instructions/investor/swap_ds_token.rs
programs/securitize-on-ramp/src/instructions/operator/swap_spl_token.rs
programs/securitize-on-ramp/src/states/asset_provider/allowance_asset_provider.rs
programs/securitize-on-ramp/src/states/asset_provider/asset_provider_core.rs
programs/securitize-on-ramp/src/states/asset_provider/minting_asset_provider.rs
programs/securitize-on-ramp/src/states/fee_manager/fee_manager_core.rs
programs/securitize-on-ramp/src/states/fee_manager/mpbs_fee_manager.rs
programs/securitize-on-ramp/src/states/nav_provider/amm_nav_provider.rs
programs/securitize-on-ramp/src/states/nav_provider/mod.rs
programs/securitize-on-ramp/src/states/nav_provider/nav_provider_core.rs
programs/securitize-on-ramp/src/states/nav_provider/standard_nav_provider.rs
programs/securitize-on-ramp/src/states/on_ramp_counter.rs
programs/securitize-on-ramp/src/states/on_ramp_state.rs
programs/securitize-on-ramp/src/states/token_type.rs
programs/securitize-on-ramp/src/utils/build_issue_tokens_cpi_data.rs
programs/securitize-on-ramp/src/utils/get_authority_signer.rs
programs/securitize-on-ramp/src/utils/get_transfer_hook_program_id.rs
programs/securitize-on-ramp/src/utils/swap_logic.rs
programs/securitize-on-ramp/src/utils/token_calculator.rs
programs/securitize-on-ramp/src/utils/transfer_maybe_hook.rs
programs/securitize-on-ramp/src/constants.rs
programs/securitize-on-ramp/src/lib.rs

// `off-ramp` changes from last audit commit 30362cf3d6b349cad72134f843808464d7477502
// to the latest commit 401146aed55b708d27982e70f4db1b2866ed37ef
//
// https://github.com/securitize-io/bc-solana-on-off-ramp-sc/compare/30362cf3d6b349cad72134f843808464d7
↪  477502...401146aed55b708d27982e70f4db1b2866ed37ef
//
// note: not including `off-ramp-cli/sdk` etc purely smart contracts

// modified files (excluded events, errors, comments-only)
programs/securitize-off-ramp/src/constants.rs
programs/securitize-off-ramp/src/instructions/admin/initialize.rs
programs/securitize-off-ramp/src/lib.rs
```

```
programs/securitize-off-ramp/src/states/off_ramp_state.rs
programs/securitize-off-ramp/src/utils/get_transfer_hook_program_id.rs
programs/securitize-off-ramp/src/utils/transfer_maybe_hook.rs

// new files
programs/securitize-off-ramp/src/instructions/admin/add_operator.rs
programs/securitize-off-ramp/src/instructions/admin/revoke_operator.rs
programs/securitize-off-ramp/src/instructions/admin/toggle_two_step_transfer.rs
programs/securitize-off-ramp/src/instructions/admin/update_asset_policy.rs
programs/securitize-off-ramp/src/instructions/admin/update_fee_manager.rs
programs/securitize-off-ramp/src/instructions/admin/update_liquidity_provider.rs
programs/securitize-off-ramp/src/instructions/admin/update_nav_provider.rs
programs/securitize-off-ramp/src/instructions/common/available_liquidity.rs
programs/securitize-off-ramp/src/instructions/common/calculate_liquidity_token_amount.rs
programs/securitize-off-ramp/src/instructions/operator/redeem_spl_token.rs
programs/securitize-off-ramp/src/instructions/user/redeem_ds_token.rs
programs/securitize-off-ramp/src/states/asset_policy.rs
programs/securitize-off-ramp/src/states/countries_restriction.rs
programs/securitize-off-ramp/src/states/fee_manager/fee_manager_core.rs
programs/securitize-off-ramp/src/states/fee_manager/mpbs_fee_manager.rs
programs/securitize-off-ramp/src/states/genesis_hash.rs
programs/securitize-off-ramp/src/states/liquidity_provider/allowance_liquidity_provider.rs
programs/securitize-off-ramp/src/states/liquidity_provider/liquidity_provider_core.rs
programs/securitize-off-ramp/src/states/nav_provider/amm_nav_provider.rs
programs/securitize-off-ramp/src/states/nav_provider/nav_provider_core.rs
programs/securitize-off-ramp/src/states/nav_provider/standard_nav_provider.rs
programs/securitize-off-ramp/src/states/token_type.rs
programs/securitize-off-ramp/src/utils/ed25519.rs
programs/securitize-off-ramp/src/utils/redemption_manager/execute_single_step_redemption.rs
programs/securitize-off-ramp/src/utils/redemption_manager/execute_two_step_redemption.rs
programs/securitize-off-ramp/src/utils/redemption_manager/mod.rs
```

6 Executive Summary

Over the course of 13 days, the Cyfrin team conducted an audit on the Solana On/Off Ramp smart contracts provided by Securitize. In this period, a total of 17 issues were found.

Securitize's Solana On-/Off-Ramp protocol provides compliant stablecoin-to-tokenized-asset conversion and redemption for regulated RWAs and public equities on Solana. The security model depends on correct CPI account validation for token minting and transfer operations, trusted NAV provider programs for pricing, operator-mediated Ed25519 signature flows for SPL token trades, and a single admin key with unrestricted configuration authority.

Our review focused on the on-ramp and off-ramp programs, covering CPI construction and account validation, Ed25519 signature verification and replay protection, fee calculation and token arithmetic, access control enforcement across all instruction paths, liquidity provider mechanics, and compliance boundary enforcement between DS Token and SPL Token flows.

The most severe risk identified was a critical vulnerability in the on-ramp's minting asset provider, where the `rwa_rbac` program ID passed via remaining accounts was not validated against the expected program address. This allowed an investor to substitute a malicious program that could mint arbitrary quantities of DS tokens by forwarding a CPI to the real `rwa_rbac` program with an inflated amount, effectively enabling infinite token minting for the cost of a minimal USDC deposit. A related medium severity finding in the same CPI path showed that the payer account for `issue_tokens` was also unvalidated, allowing investors to force the protocol's PDA or registrar authority to pay token account creation costs instead of the investor.

Additional findings concentrated around account validation gaps and configuration safety. Low severity issues included missing wallet identity verification in the DS Token subscription path when registration accounts were omitted, operator swap signatures lacking binding to specific on-ramp instances or mints, the NAV provider update instruction not enforcing executable program checks, the `available_liquidity` function reporting delegated

amounts without capping by actual token balance, the fee collector token account lacking mint and state validation during configuration, the `asset_vault` being optional during initialization despite being required for swaps, countries restriction defaulting to unrestricted on deployment, and investors being able to grief operators by triggering expensive computation before a late-failing balance check. Informational items addressed inconsistencies between on-ramp and off-ramp validation patterns, incomplete NAV provider CPI context construction, redundant instruction attributes, and the potential for country restriction changes to lock investor funds.

Summary

Project Name	Solana On/Off Ramp
Repository	bc-solana-on-off-ramp-sc
Commit	c9f35ec13465...
Audit Timeline	Feb 3rd - Feb 19th, 2026
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	0
Medium Risk	1
Low Risk	9
Informational	6
Gas Optimizations	0
Total Issues	17

Summary of Findings

[C-1] Investors can mint DsTokens out of thin air	Resolved
[M-1] Unvalidated Payer in DS Token Issue CPI Allows Protocol or Registrar to Pay Creation Fees	Resolved
[L-1] DS Token Subscription Can Proceed Without Wallet Identity Verification When Registration Accounts Are Omitted	Resolved
[L-2] Operator Swap Signatures Are Replayable and Not Bound to a Specific On-Ramp or Mints	Acknowledged
[L-3] Initialization Allows Omitted `asset_vault` Although Swaps Require It	Resolved
[L-4] Missing Executable Check for NAV Provider in `update_nav_provider`	Resolved
[L-5] Inadequate validations on `collector-token-account`.	Resolved
[L-6] Investors can grief Operators	Acknowledged
[L-7] Countries restriction defaults to empty on initialize, allowing all jurisdictions to use the protocol	Acknowledged

[L-8] Allowance Liquidity Provider Uses `delegated_amount` Without Capping by Actual Balance	Resolved
[L-9] Incomplete Rate CPI Context and Understated Minimum NAV Provider Accounts	Resolved
[I-1] On-Ramp `initialize` Should Reject Zero Genesis Hash	Resolved
[I-2] Redundant `[instruction(asset_provider)]` Attribute in On-Ramp `UpdateAssetProvider`	Resolved
[I-3] Incompatible NavProviders can be set, which could fail swaps temporarily	Resolved
[I-4] Off-Ramp `update_fee_manager` Does Not Validate `liquidity_mint`	Resolved
[I-5] Insufficient checks on liquidity amount wastes compute unnecessary	Acknowledged
[I-6] Investor's dsTokens May Become Locked After His Country Is Banned	Acknowledged

7 Findings

7.1 Critical Risk

7.1.1 Investors can mint DsTokens out of thin air

Description: For the swap logic, we are taking liquidity from investor and transferring it to custodian's liquidity token account and then we're minting the resulting amount of ds/spl tokens to investors. In `swap-ds-token` & `subscribe-ds-token` the `asset_transfer_hook_accounts`, `asset_provider_accounts` & `nav_provider_accounts` are being fetched from user supplied remaining accounts, only check that is implemented is that remaining accounts length > expected length. Investors can not spoof `nav-provider-accounts` as it is validated, they also can't fake the `asset-transfer-hook-accounts` otherwise transfer would fail and hence the whole transaction, but the `asset-provider-accounts` are intresting.. once all validation is done in respective functions, those accounts are passed in `swap-process`. Swap process transfers liquidity from users to vault and issue them ds/spl tokens via their on ramp's asset provider's `supply-to` method... In case the asset provider is `MintingAssetProvider` and token type is `DsTokens`, we are making a cpi to `rwa-rbac` program's `issue-tokens` function with passing the onramp's signing privileges(our program signed for it by providing its seeds):

```
rwa_rbac::cpi::issue_tokens(
    CpiContext::new_with_signer(
        rwa_rbac_program.to_account_info(),
        rwa_rbac::cpi::accounts::IssueTokens {
```

However the `rwa_rbac` program account was fetched from remaining accounts, and was'nt validated(to make sure its intended `rwa-rbac` program , we did'nt validate it via its key/address.

```
let rwa_rbac_program = &additional_accounts[0];
```

An Investor can simply pass his own malicious program account via remaining accounts as `rwa-rbac`. In this malicious `rwa-rbac` program attacker would implement same instruction named `issue-tokens` and the logic of this instruction is in completely hands of attacker, he can do anything in this `issue-tokens` instruction, potentially misusing the passed down signer privileges, for eg.

- the asset token vault and liquidity token vaults of onramps are owned by this authority, since authority signed this transaction, malicious user can implement `issue-tokens` such that it calls token-22's transfer checked and drains both the vaults in attacker's own token accounts. Flow of cpis: our program -> fake rwa-rbac program(attacker controlled) -> token22(for tranfer)
- A more sever attack could be produced by implementing `issue-tokens` in fake `rwa-rbac` as such: our program -> fake rwa-rbac program -> real/legit rwa-rbac program -> token22(for minting) Attacker would pass his fake program as `rwa-rbac` program to our program, we cpi to his fake program, signing with our program, passing in pda signer privileges, now attacker's program's `issue-tokens` craft a fake `cpi_data` (expected by real `rwa-rbac` program) with highly inflated token "amount" .. and cpi into real rwa-rbac program and the rwa-rbac program issues/mints the passed in " highly inflated amount" of tokens to investor.. Investors can simply mint infinite number of dsTokens this way...

Impact: Investors can mint infinite/arbitrary number of DsTokens, instead of their entitled amount.

Proof of Concept: Lets consider this scenario:

- A malicious investor identify an on-ramp that has following configuration: [asset_provider = MintingAsset-Provider] [asset_token_type = DsToken] [investor_subscription_enabled = true]
- Investor deploys a malicious program that implements [issue_tokens] instruction with the same instruction discriminator this malicious program's [issue_tokens] implementation is as follow:

1. Receives the CPI call from `securitize-on-ramp` with [on_ramp_authority] as signer
2. Ignores the original [amount] parameter
3. Constructs new CPI data with [amount = u64::MAX] (or any arbitrary large value)
4. Forwards the call to the real [rwa_rbac] program's `issue-tokens` instruction with the inflated amount

- Investor calls `swap-ds-tokens` with minimal legitimate [liquidity_amount](e.g., 10 USDC) and sets [asset_provider_accounts[0]] = their malicious program's address
- Investor's 10 usdc is transferred to vault, `swap_process` calls `MintingAssetProvider::supply_to()`, CPI to investor's malicious program (instead of real [rwa_rbac]), Malicious program CPI to real [rwa_rbac::issue_tokens] with [amount = u64::MAX], Real `rwa_rbac` mints [u64::MAX] tokens to investor's token account
- Investor receives ~u64::MAX DS tokens for the cost of 10 USDC + transaction fees

Recommended Mitigation: Validate user supplied `rwa-rbac` program Id against real/legit `rwa-rbac` program.

```
let rwa_rbac_program = &additional_accounts[0];
require_keys_eq!(
    rwa_rbac_program.key(),
    rwa_rbac::ID,  // Use the declared program ID from the imported crate
    SecuritizeOnRampError::InvalidProgram
);
```

Securitize: Fixed in 07f8e44.

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 Unvalidated Payer in DS Token Issue CPI Allows Protocol or Registrar to Pay Creation Fees

Description: In `MintingAssetProvider::supply_to` (DsToken branch), the `payer` account for the `rwa_rbac::issue_tokens` CPI is taken directly from `additional_accounts[1]` with no validation against the intended fee payer.

```
let rwa_rbac_program = &additional_accounts[0];
let payer: &_ = &additional_accounts[1];
let controller_authority = &additional_accounts[2];
// ... other accounts ...
rwa_rbac::cpi::issue_tokens(
    CpiContext::new_with_signer(
        rwa_rbac_program.to_account_info(),
        rwa_rbac::cpi::accounts::IssueTokens {
            payer: payer.to_account_info(),
            user: authority.to_account_info(),
            // ...
        },
        supply_signer,
    ),
    // ...
)?
```

This allows:

1. **PDA drain**: In `swap_ds_token` (and operator swap), the caller can set `payer` to the `on_ramp_authority` PDA. The PDA already signs via `supply_signer`; if it holds any lamports (e.g. accidental deposit or future design change), those lamports can be used to pay for token account creation and other CPI fees, effectively draining protocol-held funds.
2. **Registrar pays instead of investor**: The protocol does **not** validate that the payer is the investor. In `subscribe_ds_token`, the user supplies `asset_provider_accounts` via `remaining_accounts`. The user can set `asset_provider_accounts[1]` (the payer) to `registrar_authority`. Because `registrar_authority` is a `Signer` in the instruction, the `rwa_rbac::issue_tokens` CPI will accept them as payer and the **protocol (registrar)** will pay for the investor's token account creation (e.g. `init_if_needed`). This violates the intended design that the **user (investor)** should pay for their own account creation; the protocol is made to bear the cost.

The downstream `rwa_rbac::issue_tokens` logic typically uses `payer` for account creation (e.g. `init_if_needed` with `payer = payer`). Whoever is placed in `payer` therefore pays rent/creation; with no validation, that can be the `on_ramp PDA` or the `registrar` instead of the investor.

```
#[account(
        init_if_needed,
        payer = payer,
        associated_token::token_program = token_program,
        associated_token::mint = asset_mint,
        associated_token::authority = to,
    )]
    pub token_account: Box<InterfaceAccount<'info, TokenAccount>>,
```

Impact:

- **On-ramp authority PDA**: If the PDA ever holds lamports (e.g. mistaken transfer, future feature), any user calling `swap_ds_token` (or operator swap) can set `payer` to the PDA and cause those lamports to be spent on issue_tokens fees (e.g. token account creation). Impact is limited today if the PDA is not expected to hold balance but is a real risk if the design or usage changes.
- **Registrar as payer**: Because the payer is not validated, a user can set it to `registrar_authority` in `subscribe_ds_token` and force the protocol to pay for the investor's token account creation. This is a direct

economic cost to the protocol, contradicts the intended "user pays" design, and can be abused (e.g. high subscription volume) to shift creation costs to the protocol.

Recommended Mitigation: Make sure the `payer` is the `invester`.

Securitize: Fixed in 4382392.

Cyfrin: Verified.

7.3 Low Risk

7.3.1 DS Token Subscription Can Proceed Without Wallet Identity Verification When Registration Accounts Are Omitted

Description: The `subscribe_ds_token` instruction allows a path where all optional registration/identity accounts are None. In that branch, no CPI registration occurs and there is no on-chain wallet_identity constraint enforced before the swap executes. Unlike `swap_ds_token`, which always validates `wallet_identity` via PDA constraints, this flow can mint/transfer DS tokens to an unregistered wallet if the asset provider/token does not independently enforce identity checks.

```
(None, None, None, None, None, None, None, None, None) => {
    require_eq!(
        register_investor_cpi_data_len,
        0,
        SecuritizeOnRampError::InvalidRegisterInvestorConfig
    );
    require_eq!(
        add_levels_cpi_data_len,
        0,
        SecuritizeOnRampError::InvalidAddLevelsConfig
    );

    // All optional accounts for registering investor and adding levels are None
    require!(
        ctx.accounts.identity_metadata_registry_program.is_none()
            && ctx.accounts.investor.is_none()
            && ctx.accounts.wallet_identity.is_none()
            && ctx.accounts.policy_engine_program.is_none()
            && ctx.accounts.tracker_account.is_none()
            && ctx
                .accounts
                .event_authority_identity_metadata_registry
                .is_none()
            && ctx.accounts.policy_engine.is_none(),
        SecuritizeOnRampError::InvalidRegisterInvestorConfig
    )
}
```

- `swap_ds_token`

```
/// Check that the investor wallet is registered
#[account(
    constraint = wallet_identity.wallet == investor_wallet.key()
        @ SecuritizeOnRampError::Forbidden,
    seeds = [investor_wallet.key().as_ref(), asset_mint.key().as_ref()],
    seeds::program = identity_registry::ID,
    bump,
)]
pub wallet_identity: Box<Account<'info, identity_registry::WalletIdentity>>,
```

Impact: If DS token enforcement is not guaranteed by the asset provider or token hooks, unregistered wallets may receive DS tokens, violating compliance/identity requirements.

Recommended Mitigation: Require wallet_identity verification in subscribe_ds_token when registration is not performed, or enforce that registration CPI accounts must be supplied for DS token subscriptions.

Securitize: Fixed in 8d87df5.

Cyfrin: Verified.

7.3.2 Operator Swap Signatures Are Replayable and Not Bound to a Specific On-Ramp or Mints

Description: The operator swap signature payload (`SwapSplTokenMessage`) lacks a nonce and does not include identifiers for `on_ramp_state`, `asset_mint`, or `liquidity_mint`. As a result, a valid signature can be replayed within its deadline and reused across on-ramp instances. Because operators are trusted, this is categorized as low severity, but it still expands the blast radius if an operator key or signed payload is reused or leaked.

```
pub fn validate_investor_signature<'info>(
    ixs_account: &AccountInfo<'info>,
    expected_message: SwapSplTokenMessage,
) -> Result<()> {
    let ix_account = sysvar_instructions::get_instruction_relative(-1, ixs_account)?;

    require_gte!(
        expected_message.deadline,
        Clock::get()?.unix_timestamp,
        SecuritizeOnRampError::ExpiredSignature
    );

    utils::ed25519::validate_ed25519_ix(&ix_account)?;

    let ix_data = &ix_account.data;
    let public_key_bytes = &ix_data[16..48];

    require!(
        Pubkey::new_from_array(public_key_bytes.try_into().unwrap())
            == expected_message.investor_wallet,
        SecuritizeOnRampError::InvalidEd25519Instruction
    );

    let actual_message_hash = &ix_data[112..];

    let expected_message_bytes = expected_message.try_to_vec()?;
    let pid = crate::ID;
    let expected_message_hash =
        hashv(&[SWAP_SPL_TOKEN_TAG, pid.as_ref(), &expected_message_bytes]).to_bytes();

    require!(
        actual_message_hash == expected_message_hash,
        SecuritizeOnRampError::InvalidEd25519Instruction
    );

    Ok(())
}
```

Impact: Replay or cross-on-ramp reuse is possible, but exploitation requires a trusted operator to submit the transaction.

Recommended Mitigation: Include on_ramp_state, asset_mint, and liquidity_mint in the signed message and add a per-investor nonce (stored and consumed on-chain). This preserves trust assumptions while preventing replay across transactions or on-ramp instances.

Securitize: Acknowledged, the nonce is not included to resemble the EVM version.

7.3.3 Initialization Allows Omitted `asset_vault` Although Swaps Require It

Description: `initialize` defines `asset_vault` as an optional account (`Option<Box<InterfaceAccount<TokenAccount>>>`) and uses `init_if_needed`. This permits creating on-ramp instances without an asset_vault. However, swap instructions (`swap_spl_token`, `swap_ds_token`, `subscribe_ds_token`) require a non-optional `asset_vault` account and will fail if it does not exist. This can lead to deployed on-ramps that are unusable for swaps, especially for two-step transfers.

```rust
#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(
        mut,
        constraint = is_initializer_allowed(&admin.key())
            @ crate::errors::SecuritizeOnRampError::Forbidden,
    )]
    pub admin: Signer<'info>,

    #[account(
        mint::token_program = asset_token_program,
        mint::authority = asset_mint_authority,
    )]
    pub asset_mint: Box<InterfaceAccount<'info, Mint>>,
    /// CHECK: Mint authority for the asset mint
    pub asset_mint_authority: AccountInfo<'info>,

    #[account(
        init_if_needed,
        payer = admin,
        associated_token::mint = asset_mint,
        associated_token::authority = on_ramp_authority,
        associated_token::token_program = asset_token_program,
    )]
    pub asset_vault: Option<Box<InterfaceAccount<'info, TokenAccount>>>,
```

Impact: An on-ramp can be initialized in a state that prevents swaps.

Recommended Mitigation: Make `asset_vault` a required account in the `initialize` instruction.

Securitize: Fixed in e2807db.

Cyfrin: Verified.

7.3.4 Missing Executable Check for NAV Provider in `update_nav_provider`

Description: The on-ramp program's `update_nav_provider` instruction accepts the new NAV provider only via **instruction data** (`NavProvider`) and does not validate that the embedded program ID refers to an executable program.

```rust
pub fn update_nav_provider_handler(
    ctx: &mut Context<UpdateNavProvider>,
    nav_provider: NavProvider,
) -> Result<()> {
```

The off-ramp program's equivalent instruction accepts the new NAV provider as an **account** and enforces `#[account(executable)]`, ensuring the stored value is a valid program.

```rust
    /// New NAV provider program (must be executable)
    ///
    /// CHECK: Admin must provide a valid NAV provider program
    #[account(executable)]
    pub new_nav_provider: AccountInfo<'info>,
}
```

For On-ramp, `NavProvider` is deserialized from instruction data and Any pubkey can be written into state. The new NAV provider is passed only as instruction data and there is no account and no executable check.

Impact: This inconsistency allows the on-ramp to store an arbitrary pubkey (e.g. a non-executable account or PDA) as the NAV provider program ID, which can cause failed CPIs, unexpected behavior, or misuse if other code trusts this value.

Recommended Mitigation: Align the on-ramp with the off-ramp by ensuring the new NAV provider is validated as an executable program.

Securitize: Fixed in b4ba06a.

Cyfrin: Verified.

7.3.5 Inadequate validations on `collector-token-account`.

Description: `collector_token_account` in `initialize` and `update_fee_manager_handler` instruction lacks proper validations. When updating the [FeeManager] via the [update_fee_manager] instruction, the protocol only validates the fee percentage (numerator) but does not validate the `collector_token_account` address

The [validate()] function in [mpbs_fee_manager.rs] only checks the fee numerator:

```
impl FeeManagerTrait for MbpsFeeManager {
    fn validate(&self) -> Result<()> {
        require!(
            self.numerator <= Self::MAX_FEE_NUMERATOR,
            SecuritizeOnRampError::MaxFeeExceeded
        );
        Ok(())
    }
}
```

During swap operations, the [fee_collector_ta] is validated against the stored address in [swap_ds_token.rs] and [swap_spl_token.rs]:

```
#[account(
    mut,
    address = on_ramp_state.fee_manager.fee_collector_token_account()
        @ crate::errors::SecuritizeOnRampError::InvalidFeeCollector,
    token::mint = liquidity_mint,
    token::token_program = liquidity_token_program,
)]
pub fee_collector_ta: Box<InterfaceAccount<'info, TokenAccount>>,
```

However, this validation only occurs at swap time. If an admin sets an invalid `collector_token_account` address (e.g., wrong mint, frozen account, closed account, or non-existent account), the constraint will fail and all swaps will be blocked until the admin fixes the configuration.

Impact: Temporarily denial of service for swaps

Recommended Mitigation: Add validation of the collector_token_account in the UpdateFeeManager instruction to ensure it:

- Exists and is a valid token account
- Has the correct mint ([liquidity_mint]
- Is not frozen

Securitize: Fixed in be1ac28.

Cyfrin: Verified.

7.3.6 Investors can grief Operators

Description: In the `swap_spl_token` flow, the check for investor's token account's balance occurs at the end of the execution path inside `swap_process()`.

```
    // in `swap_logic.rs`
    // Transfer liquidity from investor
    require_gte!(
```

```
        params.investor_liquidity_ta.amount,
        params.liquidity_amount,
        SecuritizeOnRampError::InsufficientBalance
    );
```

after several computationally expensive operations have already been performed:

- Pause state check
- Token type validation
- Minimum subscription amount check
- Ed25519 signature verification (expensive - [validate_investor_signature])
- Remaining accounts validation
- Fee calculation
- AMM [execute_buy_base] CPI call (expensive)
- Asset amount calculation

Finally: Inside `swap_process()` after checking for slippage & calculating `liquidity_amount_excluding_fee`, it finally checks that Investor's token balance is greater than liquidity amount, an malicious investor can exploit this ordering of operations and expensive checks to grief operators(becuase operator is signer, he pays for transaction fees) and as the CU raises, the fee is raised too. These many checkes and cpis increase cu heavily. Operator bears this transaction fee cost.

Impact: This setup can be used by malicious investors to grief operators by making them pay large transaction fees and making the transaction fail on purpose.

Recommended Mitigation: Move the balance check to the beginning of `swap_spl_token_handler` before expensive operations:

```
pub fn swap_spl_token_handler<'info>(
    ctx: &Context<'_, '_, '_, 'info, SwapSplToken<'info>>,
    liquidity_amount: u64,
    min_out_amount: u64,
    deadline: i64,
    asset_provider_accounts_count: u8,
    nav_provider_params: &NavProviderParams,
) -> Result<()> {
    let on_ramp_state = &ctx.accounts.on_ramp_state;

    require!(!on_ramp_state.is_paused, SecuritizeOnRampError::Paused);

    // Early balance check - fail fast before expensive operations
    require_gte!(
        ctx.accounts.investor_liquidity_ta.amount,
        liquidity_amount,
        SecuritizeOnRampError::InsufficientBalance
    );

    require!(
        on_ramp_state.asset_token_type == crate::states::TokenType::SplToken,
        SecuritizeOnRampError::InvalidTokenType
    );

    // ...existing code...
```

Note: Also make sure the `investor_asset_ta` is not frozen upfront, otherwise a faulty investor can execute same griefing with his frozen token account(spl token account).

Securitize: Acknowledged.

7.3.7 Countries restriction defaults to empty on initialize, allowing all jurisdictions to use the protocol

Description: When a new off-ramp state is created via `initialize`, `countries_restriction` is hardcoded to `CountriesRestriction::default()` (an all-zero bitmap). No country is therefore restricted until an admin explicitly calls `update_countries_restriction`.

```
let off_ramp_state_inst = OffRampState {
    admin: ctx.accounts.admin.key(),
    asset_mint: ctx.accounts.asset_mint.key(),
    asset_policy,
    asset_token_type,
    bump: ctx.bumps.off_ramp_state,

    id: counter,
    is_paused: false,
    off_ramp_authority_bump: ctx.bumps.off_ramp_authority,
    nav_provider,
    liquidity_mint: ctx.accounts.liquidity_mint.key(),
    fee_manager,
    countries_restriction: CountriesRestriction::default(),
    operators: vec![],
    two_step_transfer: false,
    liquidity_provider,
};
```

`CountriesRestriction` is a 32-byte bitmap; the default is all zeros:

```
/// Bitmap for restricting up to 256 countries
#[derive(AnchorDeserialize, AnchorSerialize, Clone, Debug, InitSpace, Default)]
pub struct CountriesRestriction([u8; 32]);
```

With all bits zero, `is_restricted(idx)` is false for every country index:

```
/// Returns true if the country is restricted
pub fn is_restricted(&self, idx: u8) -> bool {
    let byte = self.0[(idx / 8) as usize];
    let bit = idx % 8;
    (byte & (1 << bit)) != 0
}
```

User redemption enforces the restriction here:

```
require!(
    !off_ramp_state
        .countries_restriction
        .is_restricted(redeemer_country),
    SecuritizeOffRampError::RestrictedCountry,
);
```

So from the first `initialize` until the first `update_countries_restriction` (and if that call is forgotten or delayed, indefinitely), no country is restricted and users from any jurisdiction can redeem.

Impact: Compliance / regulatory risk: If the protocol is required to restrict certain countries from launch (e.g., sanctions or licensing), the default behavior is non-compliant until an admin updates the bitmap.

Recommended Mitigation: Add an optional (or required) argument to the `initialize` instruction so the deployer can set the initial bitmap in the same transaction.

Securitize: Acknowledged, We wouldn't want to make the init logic heavier and tightly coupled with compliance config. And the admin, can always combine the initialize and update_countries_restriction instructions into one transaction if needed.

7.3.8 Allowance Liquidity Provider Uses `delegated_amount` Without Capping by Actual Balance

Description: For `AllowanceLiquidityProvider`, `available_liquidity` returns the source token account's `delegated_amount` only.

```
        let source_token_account =
            TokenAccount::try_deserialize(&mut &source_token_account_info.data.borrow()[..])?;

        require_keys_eq!(
            source_token_account.delegate.unwrap_or_default(),
            expected_delegate.key(),
            SecuritizeOffRampError::InvalidLiquidityProviderConfiguration
        );

        Ok(source_token_account.delegated_amount)
```

In SPL Token, `delegated_amount` is not reduced when tokens are transferred out of the account after approval via `self-transfer`. Thus the reported "available liquidity" can exceed the account's current balance. This leads to inflated liquidity visibility and redemptions that can fail at transfer time when the LP's balance is lower than the reported allowance.

```
                if !self_transfer {
                    source_account.delegated_amount = source_account
                        .delegated_amount
                        .checked_sub(amount)
                        .ok_or(TokenError::Overflow)?;
                    if source_account.delegated_amount == 0 {
                        source_account.delegate = COption::None;
                    }
                }
```

In addition, `calculate_effective_liquidity_amount` returns the requested amount unchanged and does not consider the real token balance or available liquidity.

Impact:

- **Incorrect liquidity reporting**: Off-ramp and integrators may show "available liquidity" equal to `delegated_amount` even when the LP's balance is lower, e.g. after the LP transferred tokens elsewhere.
- **Failed redemptions**: Users or operators may initiate redemptions for amounts that then fail at transfer because the source account has insufficient balance. This causes failed transactions and poor UX (and can be seen as a form of DoS for those redemption attempts).

Recommended Mitigation: Cap `available_liquidity` by current balance, return something like `Ok(source_token_account.delegated_amount.saturating_min(source_token_account.amount))`.

Securitize: Fixed in cdd059f.

Cyfrin: Verified.

7.3.9 Incomplete Rate CPI Context and Understated Minimum NAV Provider Accounts

Description: The on-ramp and off-ramp programs invoke NAV provider `rate` instructions via CPI without signing the call with the `on_ramp_authority` or `off_ramp_authority` PDA. The CPI is constructed as an unsigned context:

```
let get_rate_ctx = CpiContext::new(
    nav_provider_program.to_account_info(),
    nav_provider_interface::cpi::accounts::Rate {
        asset_mint: asset_mint.to_account_info(),
        nav_provider_state: nav_provider_state.to_account_info(),
    },
)
.with_remaining_accounts(nav_provider_accounts[2..].to_vec());
```

Since the NAV provider program is within the protocol's trust boundary and the admin controls which program is set, this is currently safe. However, if a future NAV provider needs to authenticate the caller (e.g. to restrict rate queries to authorized ramp programs, enforce per-caller rate limits, or distinguish between on-ramp and off-ramp callers for different pricing behavior), the current unsigned CPI pattern would not support this without a protocol upgrade.

The same pattern is used in the off-ramp's `get_rate` and AMM NAV provider paths (`execute_buy_base`, `execute_sell_base`, `quote_buy_base`, `quote_sell_base`), none of which sign with the ramp authority PDA.

Impact: NAV provider programs cannot verify the identity of the calling ramp program. While all current NAV providers are within the trust boundary and do not require caller authentication, this limits the extensibility of the NAV provider interface. If a future NAV provider needs to gate access or vary behavior by caller, the interface would need to be updated across both programs.

Recommended Mitigation: Sign NAV provider CPI calls with the `on_ramp_authority` / `off_ramp_authority` PDA and include the authority account in the Rate interface struct. This allows NAV providers to optionally verify the caller without requiring it:

```
let get_rate_ctx = CpiContext::new_with_signer(
    nav_provider_program.to_account_info(),
    nav_provider_interface::cpi::accounts::Rate {
        asset_mint: asset_mint.to_account_info(),
        nav_provider_state: nav_provider_state.to_account_info(),
        caller_authority: on_ramp_authority.to_account_info(),
    },
    &[&on_ramp_authority_seeds],
)
.with_remaining_accounts(nav_provider_accounts[2..].to_vec());
```

Securitize: Fixed in c8dd8d9.

Cyfrin: Verified.

7.4 Informational

7.4.1 On-Ramp `initialize` Should Reject Zero Genesis Hash

Description: The on-ramp `initialize` instruction uses `[0; 32]` as the sentinel for "genesis hash PDA not yet set" but does not reject the same value when it is passed as the `genesis_hash` instruction argument.

```
    // Genesis hash is a singleton config used for signature domain separation.
    // It is set once on first initialize; subsequent initializes must match.
    if ctx.accounts.genesis_hash.hash == [0; 32] {
        ctx.accounts.genesis_hash.set_inner(GenesisHash {
            hash: genesis_hash,
            bump: ctx.bumps.genesis_hash,
        });
    } else {
        require!(
            ctx.accounts.genesis_hash.hash == genesis_hash,
            crate::errors::SecuritizeOnRampError::GenesisHashMismatch
        );
    }
```

There is no upfront check that the **instruction argument** `genesis_hash != [0; 32]`, so the "wrong" outcome (storing zero) is not prevented.

```
pub fn initialize_handler(
    ctx: &mut Context<Initialize>,
    fee_manager: crate::FeeManager,
    asset_provider: crate::AssetProvider,
    nav_provider: crate::NavProvider,
    custodian_wallet: Pubkey,
    genesis_hash: [u8; 32],
) -> Result<()> {
```

As a result, the first `initializer` can set the singleton genesis hash to an all-zero value. This value is then used as the cluster-specific domain separator for signature verification (e.g., in `swap_spl_token`). Moreover, this design does not guarantee that the `genesis_hash` remains consistent across all on-ramp states.

Impact: Failing to reserve `[0; 32]` as "uninitialized only" blurs the meaning of the sentinel.

Recommended Mitigation: Reject the zero hash at the start of the handler so it is reserved for "uninitialized" only and cannot be stored as the genesis hash.

Securitize: Fixed in 5b035d1.

Cyfrin: Verified.

7.4.2 Redundant `#[instruction(asset_provider)]` Attribute in On-Ramp `UpdateAssetProvider`

Description: The `UpdateAssetProvider` accounts struct declares the instruction argument `asset_provider` via `#[instruction(asset_provider: crate::AssetProvider)]`, but no account constraint (e.g. `constraint`, `seeds`, or `has_one`) references this variable.

```
#[derive(Accounts)]
#[instruction(asset_provider: crate::AssetProvider)]
pub struct UpdateAssetProvider<'info> {
    pub admin: Signer<'info>,

    #[account(
        mut,
        has_one = admin @ SecuritizeOnRampError::Forbidden,
        seeds = [ON_RAMP_STATE_SEED, on_ramp_state.id.to_le_bytes().as_ref()],
        bump = on_ramp_state.bump,
    )]
```

```
    pub on_ramp_state: Box<Account<'info, OnRampState>>,
}
```

The argument is only used in the handler. The attribute is therefore redundant and can be removed for clarity. So the `#[instruction(asset_provider: ...)]` declaration is redundant.

Impact: The relevant code is redundant and can be removed for code quality.

Recommended Mitigation: Remove the unused attribute.

Securitize: Fixed in b29a57.

Cyfrin: Verified.

7.4.3 Incompatible NavProviders can be set, which could fail swaps temporarily

Description: The protocol has two distinct swap token types, `Dstoken` and `SplToken`. Each tokenType is coupled with specific `NavProvider`. DsTokens require StandardNavProvider where splTokens require AmmNavProvider.. If wrong NavProviders are set, it throws errors: In case of ds tokens' swaps

```
        NavProvider::AmmNavProvider(_) => {
            return err!(SecuritizeOnRampError::UnsupportedNavProvider);
        }
```

In case of spl tokens' swaps:

```
        NavProvider::StandardNavProvider(_) => {
            return err!(SecuritizeOnRampError::UnsupportedNavProvider)
        }
```

However, the `update_nav_provider_handler` instruction does'nt make sure the passed in `NavProviders` are compatible with the tokenType of particular onramp, admin can mistakenly set incompitable NavProviders and hence swaps would fail until type is fixed.

Note: similarly `update_asset_provider_handler` lacks incompatibility validation too

Impact: Temporarily swap failures.

Recommended Mitigation: Add compatibility validation:

```
pub fn update_nav_provider_handler(
    ctx: &mut Context<UpdateNavProvider>,
    nav_provider: NavProvider,
) -> Result<()> {
    let on_ramp_state = &mut ctx.accounts.on_ramp_state;

    let old_nav_provider = on_ramp_state.nav_provider;

    require!(
        old_nav_provider != nav_provider,
        SecuritizeOnRampError::NoChange
    );

    // Validate NavProvider is compatible with asset_token_type
    match (&on_ramp_state.asset_token_type, &nav_provider) {
        (TokenType::DsToken, NavProvider::AmmNavProvider(_)) => {
            return err!(SecuritizeOnRampError::UnsupportedNavProvider);
        }
        (TokenType::SplToken, NavProvider::StandardNavProvider(_)) => {
            return err!(SecuritizeOnRampError::UnsupportedNavProvider);
        }
        _ => {}
    }
```

```
    on_ramp_state.nav_provider = nav_provider;

    // ...existing code...
    Ok(())
}
```

Securitize: Fixed in d14f30e.

Cyfrin: Verified.

7.4.4 Off-Ramp `update_fee_manager` Does Not Validate `liquidity_mint`

Description: In the on-ramp program, `update_fee_manager` explicitly binds the instruction to a `liquidity_mint` by requiring `on_ramp_state` to have `has_one = liquidity_mint` and by passing a `liquidity_mint` account.

```
    #[account(
        mut,
        has_one = liquidity_mint @ SecuritizeOnRampError::InvalidMint,
        has_one = admin @ SecuritizeOnRampError::Forbidden,
        seeds = [ON_RAMP_STATE_SEED, on_ramp_state.id.to_le_bytes().as_ref()],
        bump = on_ramp_state.bump,
    )]
    pub on_ramp_state: Box<Account<'info, OnRampState>>,

    #[account(
        mint::token_program = liquidity_token_program,
    )]
    pub liquidity_mint: Box<InterfaceAccount<'info, Mint>>,

    pub liquidity_token_program: Interface<'info, TokenInterface>,
```

The off-ramp program's `update_fee_manager` does not perform any `liquidity_mint`-based check, even though `OffRampState` also has a `liquidity_mint` field. This inconsistency can make it easier to target the wrong state by mistake and weakens alignment between the two programs.

```
    #[account(
        mut,
        has_one = admin @ SecuritizeOffRampError::Forbidden,
        seeds = [OFF_RAMP_STATE_SEED, off_ramp_state.id.to_le_bytes().as_ref()],
        bump = off_ramp_state.bump,
    )]
    pub off_ramp_state: Box<Account<'info, OffRampState>>,
```

Impact: On-ramp and off-ramp diverge in how they constrain `update_fee_manager`. Requiring the liquidity mint account and `has_one = liquidity_mint` would make the "which state for which mint" invariant explicit and align off-ramp with on-ramp and with other off-ramp instructions.

Recommended Mitigation: Align off-ramp with on-ramp and with other off-ramp instructions.

Securitize: Fixed in bbcdfe0.

Cyfrin: Verified.

7.4.5 Insufficient checks on liquidity amount wastes compute unnecessary

Description: In both `[redeem_ds_token_handler]` and `[redeem_spl_token_handler]`, after computing `[liquidity_amount]` from the NAV rate and asset amount, the code proceeds directly into `[redemption_manager::redeem()]` without first verifying that the liquidity provider's `[source_token_account]` holds sufficient funds (or has sufficient delegated allowance) to fulfill the redemption.

```
let liquidity_amount = utils::token_calculator::calculate_liquidity_amount(
    asset_amount,
```

```
    rate,
    ctx.accounts.asset_mint.decimals,
    ctx.accounts.liquidity_mint.decimals,
)?;

require_gt!(liquidity_amount, 0, SecuritizeOffRampError::ZeroAmount);

// No check that the liquidity provider can actually fulfill `liquidity_amount`

let (fee_amount, user_supplied_amount) = redemption_manager::redeem(RedemptionParams {
    // ...
})?;
```

In the two-step redemption flow [execute_two_step_redemption.rs], this means the redeemer's asset tokens are first transferred to the [asset_vault], and then the subsequent [supply_to] call from the liquidity provider fails due to insufficient balance or delegation. While Solana's transaction atomicity ensures the entire transaction reverts (no funds are lost), the failure occurs deep inside the CPI chain rather than being caught early with a clear error. Which costs the operator or investor unnecessary extra computes

Impact: The late failure wastes compute units, produces SPL Token transfer errors instead of a clear InsufficientLiquidity error, and degrades user experience. For operator-mediated SPL token redemptions, repeated late failures can cause signature expiration, forcing the redeemer through the off-chain signing flow again.

Recommended Mitigation: Add an early liquidity sufficiency check immediately after computing [liquidity_amount], before calling [redemption_manager::redeem()]. The [liquidity_provider_accounts] (which include the [source_token_account]) are already available at this point:

```
let liquidity_amount = utils::token_calculator::calculate_liquidity_amount(
    asset_amount,
    rate,
    ctx.accounts.asset_mint.decimals,
    ctx.accounts.liquidity_mint.decimals,
)?;

require_gt!(liquidity_amount, 0, SecuritizeOffRampError::ZeroAmount);

// Add early liquidity check
let available = off_ramp_state
    .liquidity_provider
    .available_liquidity(off_ramp_state, liquidity_provider_accounts)?;
require_gte!(
    available,
    liquidity_amount,
    SecuritizeOffRampError::InsufficientLiquidity
);

let (fee_amount, user_supplied_amount) = redemption_manager::redeem(RedemptionParams {
    // ...existing code...
})?;
```

Securitize: Acknowledged.

7.4.6 Investor's dsTokens May Become Locked After His Country Is Banned

Description: The off-ramp program enforces country-based restrictions during DS token redemption via the [countries_restriction] bitmap stored in [OffRampState]. Before processing any redemption, [redeem_ds_token_handler] reads the investor's country from their on-chain [IdentityAccount] and verifies it is not restricted:

```
let redeemer_country = ctx.accounts.identity_account.country;

require!(
    !off_ramp_state
```

```
        .countries_restriction
        .is_restricted(redeemer_country),
    SecuritizeOffRampError::RestrictedCountry,
);
```

However, the on-ramp program, which handles the inverse flow (investor sends liquidity tokens to receive asset tokens) has no country restriction mechanism at all. The [OnRampState] struct lacks a [countries_restriction] field entirely:

```
pub struct OnRampState {
    pub id: u64,
    pub admin: Pubkey,
    pub asset_mint: Pubkey,
    pub asset_token_type: TokenType,
    pub liquidity_mint: Pubkey,
    pub nav_provider: NavProvider,
    pub is_paused: bool,
    pub fee_manager: FeeManager,
    pub custodian_wallet: Pubkey,
    pub bump: u8,
    pub on_ramp_authority_bump: u8,
    pub min_subscription_amount: u64,
    pub investor_subscription_enabled: bool,
    pub two_step_transfer: bool,
    pub asset_provider: AssetProvider,
    pub operators: Vec<Pubkey>,
    // No countries_restriction field
}
```

There are no checks implemented for checking investor's country inside all three swap functions:

- for `swap_spl_tokens` the operator signs, s**o we can trust here that operator would not sign for investors from banned countries**
- for `subscribe_ds_tokens`, registrar authority signs, so we can assume above here too
- but for `swap_ds_tokens` only investor signs the transaction while providing his `wallet_identity` account, only check that is made is that `wallet_identity` account belongs to investor signing the transaction, the on-ramp program does not enforce country restrictions when processing `swap_ds_tokens`:

```
    /// Check that the investor wallet is registered
    #[account(
        constraint = wallet_identity.wallet == investor_wallet.key()
            @ SecuritizeOnRampError::Forbidden,
        seeds = [investor_wallet.key().as_ref(), asset_mint.key().as_ref()],
        seeds::program = identity_registry::ID,
        bump,
    )]
    pub wallet_identity: Box<Account<'info, identity_registry::WalletIdentity>>,
```

Although `swap_ds_tokens` requires that the investor has previously called `subscribe_ds_tokens` (since a `wallet_identity` account must exist), there is no country re-validation during the swap flow.

This creates the following state transition inconsistency:

1. An investor from a non-restricted country calls `subscribe_ds_tokens` and successfully creates their `wallet_identity` or even hold some DS tokens.
2. At a later time, the admin updates the off-ramp `countries_restriction` bitmap and bans the investor's country.
3. The old investor can still call `swap_ds_tokens` on the on-ramp, since no country restriction is enforced there.
4. The investor acquires additional DS tokens.

5. When attempting to redeem through the off-ramp, `redeem_ds_token_handler` rejects the transaction with `RestrictedCountry`.

Generally, if the investor already held DS tokens before the country was banned, those tokens immediately become non-redeemable. As a result, DS tokens can become economically locked: the investor holds a token but cannot exit via the protocol's designated redemption mechanism which is more of a design flaw. This lock persists unless the admin later removes the country restriction on the off-ramp, which may contradict the regulatory rationale for imposing the restriction.

Impact: Investors whose country becomes restricted after registration:

- Can continue acquiring DS tokens through `swap_ds_tokens`
- Cannot redeem those tokens through the off-ramp
- May have pre-existing DS tokens that become permanently non-redeemable

Investors from restricted countries can acquire asset tokens through the on-ramp that they are unable to redeem through the off-ramp. This results in locked funds from the investor.

Recommended Mitigation: Add a [countries_restriction] field (identical [CountriesRestriction] bitmap) to [OnRampState] and enforce country checks in all investor-facing on-ramp instructions..

Securitize: Acknowledged; this is expected behavior and aligns with the requirements.